Project Packages and Nokia Integration Requirements

AI-Based Power Saving in Optical Networks

This document lists the packages currently declared by the project and the packages that may be required after the Nokia telemetry interface is confirmed.

Current backend Python packages

Package	Purpose
fastapi >= 0.100.0	Backend REST API
uvicorn >= 0.22.0	Runs the FastAPI server
pydantic >= 2.0.0	Request, response, and telemetry models
numpy >= 1.24.0	Numerical calculations
scikit-learn >= 1.2.0	Machine-learning calculations
networkx >= 3.0	Optical network topology graphs
qiskit-optimization >= 0.7.0	Quantum and optimization algorithms

Current frontend packages

Runtime dependencies

Package	Purpose
next	Next.js web framework
react / react-dom	React user interface and browser rendering
recharts	Telemetry and analytics charts
lucide-react	UI icons
clsx	Conditional CSS class names
tailwind-merge	Merges Tailwind CSS classes

Development dependencies

Package	Purpose
typescript	TypeScript support
@types/node	Node.js type definitions
@types/react / @types/react-dom	React and React DOM type definitions
tailwindcss	Tailwind CSS
postcss	CSS processing
autoprefixer	Browser CSS prefixes

Possible Nokia integration packages

Install only the package matching the interface confirmed by the Nokia team. Do not install every option.

Confirmed interface	Likely package
JSON or CSV file	No extra package normally required
SNMP	pysnmp
NETCONF	ncclient
REST API or RESTCONF	httpx
MQTT	paho-mqtt
Kafka	confluent-kafka or kafka-python
SQL database	Database-specific driver, possibly SQLAlchemy
gNMI	Nokia-approved or compatible gNMI client

Information to request from the mentor

1. Confirm whether telemetry comes from the NMS or directly from the NE.
2. Provide the exact protocol, port, API endpoint, and connection method.
3. Explain how NMS IP 100.124.251.32 and NE IP 192.124.242.190 should be used.
4. Provide VPN, firewall, routing, and IP allowlisting requirements.
5. Provide read-only test access through the approved secure channel.
6. Provide Nokia API documentation, MIB files, YANG models, or interface documentation.
7. Provide device and transponder inventory with IDs, models, locations, ports, wavelengths, and capacities.
8. Provide telemetry field names, units, valid ranges, and timestamp rules.
9. Provide sanitized machine-readable samples for normal and faulty conditions.
10. Provide alarm and fault-code mappings.
11. Confirm whether the system is monitoring-only or must execute hardware actions.
12. Provide a test device and rollback procedure for hardware commands.
13. Provide refresh rate, expected scale, retention, and deployment requirements.

Recommended integration sequence

Start with read-only telemetry. Add a Nokia adapter that converts the vendor fields into the project's existing telemetry model. Validate units, timestamps, freshness, and alarm behavior before enabling any hardware control. Use operator approval and a test transponder before production actions.

Project files: backend/requirements.txt and frontend/package.json